

Processos e comunicação entre processos

fork, memória independente e pipe na prática

Weslei Ferreira Santos

5 de outubro de 2026

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

min	Etapa
0 a 8	Programa e processo
8 a 20	Uma chamada de <code>fork()</code>
20 a 33	<code>fork()</code> dentro do laço: <code>fork2.c</code>
33 a 45	Memória independente e <code>waitpid</code>
45 a 53	Checagem em duplas
53 a 65	Pipe: canal, pontas e bloqueio
65 a 75	Aplicação com vários processos: <code>pipe2.c</code>
75 a 87	Prática: filho envia, pai recebe
87 a 90	Bilhete de saída

Como um processo recebe o resultado calculado por outro?

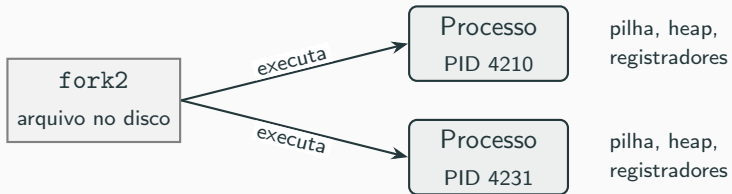
Ao final, você deve conseguir:

- identificar os retornos de `fork()` no pai e no filho;
- explicar por que cada processo tem a sua própria memória;
- desenhar a árvore de processos de um laço com `fork()`;
- completar uma comunicação filho → pai com `pipe()`.

Programa e processo

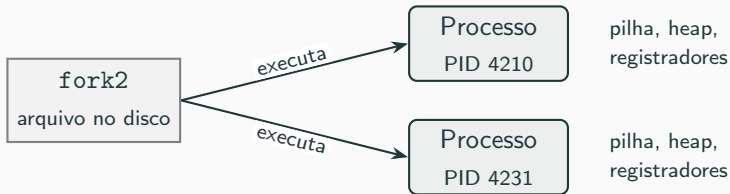
Um programa ou dois processos?

Se eu executar o mesmo programa em dois terminais, tenho um programa ou dois processos?



Um programa ou dois processos?

Se eu executar o mesmo programa em dois terminais, tenho um programa ou dois processos?



Definição

O arquivo contém as instruções. Cada execução é um **processo**: uma instância com PID, registradores, pilha, heap e recursos próprios.

Se uma dessas execuções altera uma variável,
a outra *necessariamente* observa a alteração?

Anote sua hipótese. Vamos testá-la com `fork2.c`.

O que o SO guarda sobre cada processo

Para cada processo, o núcleo mantém um registro (o *bloco de controle do processo*):

PID	Número que identifica o processo enquanto ele existe.
PPID	PID do processo que o criou (o pai).
Estado	Executando, pronto, bloqueado esperando algo, terminado.
Contexto	Registradores, incluindo onde o programa parou; salvos quando ele sai da CPU.
Memória	O espaço de endereçamento: código, dados, heap e pilha.
Descritores	Arquivos, terminais e pipes que o processo tem abertos.

No terminal, `ps -f` mostra PID e PPID; `ps tree -p` mostra quem criou quem.

A memória de um processo



- **Pilha:** variáveis locais e chamadas de função.
- **Heap:** malloc, calloc.
- **Dados:** variáveis globais e static.
- **Código:** as instruções do programa.

Endereços virtuais

Os endereços que o programa usa são **virtuais**. O SO e o hardware traduzem cada um para uma posição física, e a tradução é **diferente para cada processo**. Visão simplificada: omite bibliotecas compartilhadas e o sorteio de endereços (ASLR).

Uma chamada de `fork()`

Uma chamada, dois caminhos

```
pid_t pid = fork();
if (pid == -1) {
    /* falha: nenhum filho criado */
} else if (pid == 0) {
    /* este caminho executa no filho */
} else {
    /* executa no pai; pid = filho */
}
/* pai e filho podem chegar aqui */
```

Quem	fork() retorna
Filho	0
Pai	PID do filho
Falha	-1

O filho não recomeça o main

Pai e filho continuam **depois** da chamada, cada um com seu retorno.

O que o filho recebe no `fork()`

Igual ao pai (cópia)

- Toda a memória: código, globais, heap, pilha.
- Valores das variáveis *naquele instante*.
- Registradores: continua no mesmo ponto.
- Descritores abertos (arquivos, pipes).

Diferente do pai

- PID novo.
- PPID = PID do pai.
- Retorno de `fork()`: 0.

Cópia na escrita (*copy-on-write*)

Pai e filho dividem as páginas até um deles escrever; só então a página é copiada. O efeito é o de uma cópia completa: **o que um escreve, o outro não vê.**

getpid() e a ordem de execução

- getpid() retorna o PID do processo que *está executando*.
- O valor que fork() devolve ao pai identifica o **filho**, não o próprio pai.

Previsão: o pai necessariamente executa o próximo printf antes do filho?

getpid() e a ordem de execução

- getpid() retorna o PID do processo que *está executando*.
- O valor que fork() devolve ao pai identifica o **filho**, não o próprio pai.

Previsão: o pai necessariamente executa o próximo printf antes do filho?

Resposta

Não. A ordem depende do escalonamento e da sincronização que o programa estabelecer.

Vendo os dois retornos: fork_basico.c

```
pid_t pid = fork();
if (pid == 0) {
    printf("Filho: ... meu PID %d; meu pai %d\n", getpid(), getppid());
    fflush(stdout); _exit(EXIT_SUCCESS);
}
printf("Pai: fork retornou %d; meu PID %d\n", pid, getpid());
```

```
$ make run-fork-basico
./build/fork-basico
Antes do fork: PID 4210
Filho: fork retornou 0; meu PID 4211; meu pai 4210
Pai: fork retornou 4211; meu PID 4210
Pai: filho 4211 terminou com codigo 0
```

O pai recebe 4211, que é o PID do filho; o pai do filho é 4210. Os PIDs mudam a cada execução.

Um detalhe de C: printf, buffers e _exit

- printf guarda o texto num **buffer na memória do processo** e só entrega ao SO depois (quebra de linha no terminal, buffer cheio ou fim do programa).
- O buffer faz parte da memória, então é **copiado pelo fork()**: texto pendente pode sair duas vezes.
- _exit encerra **sem esvaziar** o buffer. exit e return esvaziam, inclusive o que foi herdado do pai.

Regra prática

fflush(stdout) antes do fork(). No filho que termina no meio do código do pai, fflush(stdout) e depois _exit.

`fork()` dentro do laço

fork2.c: o trecho central

```
int *v = calloc((size_t)tamanho, sizeof *v);    /* tamanho = 3 */
pid_t original = getpid();
for (int i = 0; i < tamanho; ++i) {
    pid_t pid = fork();
    if (pid == -1) { /* tratamento de falha */ }
    if (pid == 0) v[i] = i + 5;
}
```

Dois minutos, no papel

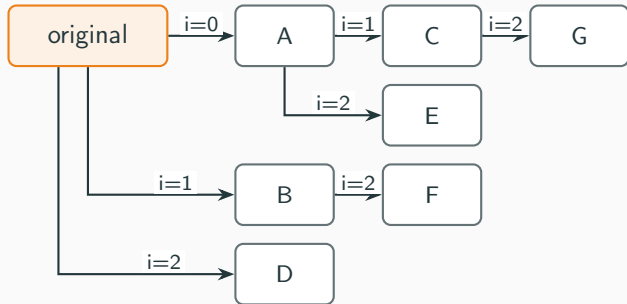
Com tamanho = 3, quantos processos existem ao todo? Desenhe quem cria quem.

Lendo fork2.c por partes

Trecho	O que faz
<code>calloc(tamanho, ...)</code>	Reserva o vetor já zerado : [0, 0, 0].
<code>original = getpid()</code>	Guarda o PID de quem começou. Os filhos herdam o número, mas só um processo tem <code>getpid() == original</code> .
laço com <code>fork()</code>	Em cada volta, todo processo vivo chama <code>fork()</code> . O filho escreve <code>v[i] = i + 5</code> .
laço com <code>waitpid</code>	Cada processo espera os seus filhos diretos, até não restar nenhum (ECHILD).
<code>if (getpid()==original)</code>	Só o processo original soma o vetor e imprime.

Pergunta-chave: **o filho sai do laço depois de nascer?** Não: ele continua as voltas que faltam.

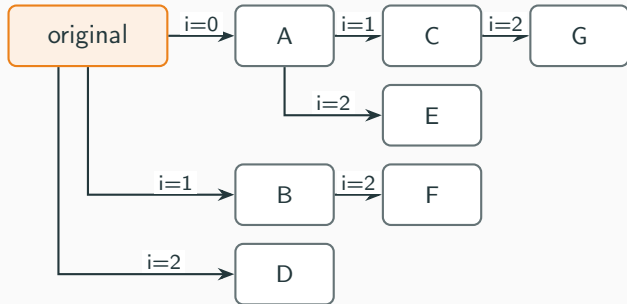
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1

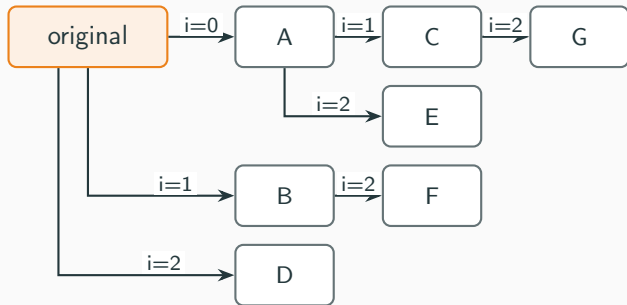
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- i=0: 2

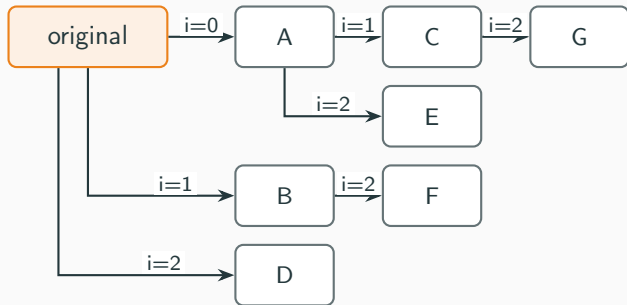
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- $i=0$: 2
- $i=1$: 4

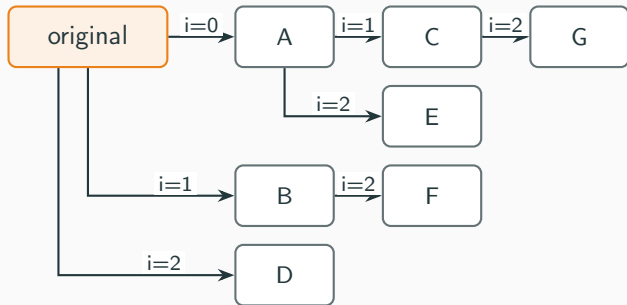
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- $i=0$: 2
- $i=1$: 4
- $i=2$: 8

A árvore de fork2.c



Caminhos ao fim de cada iteração

- início: 1
- $i=0$: 2
- $i=1$: 4
- $i=2$: 8

Todos continuam o laço: **8 processos** ao todo, **7 novos**,
 $1 + 2 + 4 = 7$ chamadas executadas.

Dois cuidados com essa contagem

- 8 conta os processos criados *ao longo da execução*.
Não significa que os oito estejam vivos ao mesmo tempo.
- O código tem **uma** chamada de `fork()`; a árvore executa **sete**.

E se cada filho saísse do laço logo após nascer?

Dois cuidados com essa contagem

- **8** conta os processos criados *ao longo da execução*.
Não significa que os oito estejam vivos ao mesmo tempo.
- O código tem **uma** chamada de `fork()`; a árvore executa **sete**.

E se cada filho saísse do laço logo após nascer?

Resposta

Só o original completaria as três iterações: **4 processos** ao todo (o original e três filhos).

Memória independente

Previsão antes de executar

Cada filho escreve em $v[i]$. O pai soma o vetor no final.

Qual soma o pai imprime? **0?** **18?** outro valor?

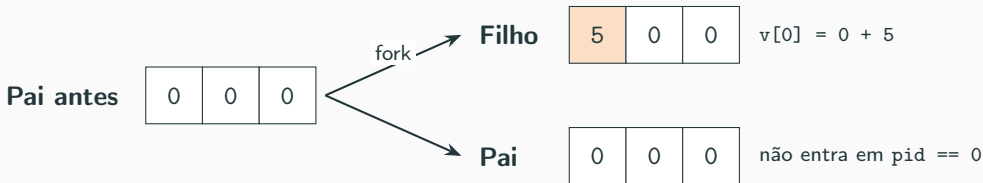
Previsão antes de executar

Cada filho escreve em `v[i]`. O pai soma o vetor no final.

Qual soma o pai imprime? 0? 18? outro valor?

```
$ make run-fork  
./build/fork2  
Soma total no pai: 0
```

Cada processo escreve na própria cópia



- `calloc` começa com três zeros.
- O filho altera a **sua** memória; o vetor do pai continua `[0, 0, 0]`.
- `getpid() == original`: só o processo original imprime a soma.

O vetor de cada um dos oito processos

Proc.	Criado por	Vetor	Soma
original	início	[0,0,0]	0
A	original, i=0	[5,0,0]	5
B	original, i=1	[0,6,0]	6
D	original, i=2	[0,0,7]	7
C	A, i=1	[5,6,0]	11
E	A, i=2	[5,0,7]	12
F	B, i=2	[0,6,7]	13
G	C, i=2	[5,6,7]	18

- O filho herda o vetor **como estava no pai** no momento do `fork()`.
- Depois escreve só a sua posição.
- O 18 existe, mas só em **G**, que não é o original e não imprime.

Mesmo endereço, valores diferentes

```
int x = 10;
pid_t pid = fork();
if (pid == 0) { x = 99; printf("Filho: x = %d em %p\n", x, (void *)&x); ... }
printf("Pai:    x = %d em %p\n", x, (void *)&x);    /* depois de esperar o filho */
```

```
$ make run-memoria
./build/memoria-independente
Filho: x = 99 em 0x7ffe72f6ea88
Pai:    x = 10 em 0x7ffe72f6ea88
```

Mesmo endereço não é mesma memória

O endereço é **virtual**: cada processo o interpreta no seu próprio espaço, e o SO o leva a posições físicas diferentes.

O que waitpid faz

- Espera um filho terminar.
- Coleta o estado de término.
- Cada processo coleta seus **filhos diretos**.

O que ele não faz

- Não copia o vetor do filho.
- Não cria memória compartilhada.
- Não transporta resultados.

Por que a soma não virou 18?

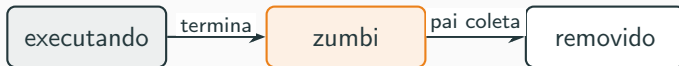
O pai já espera os filhos, mas esperar coordena o *término*. Nem sleep nem waitpid tornam o heap compartilhado.

```
pid_t waitpid(pid_t pid, int *status, int opcoes);
```

- `pid > 0`: espera esse filho; `-1`: qualquer filho.
- `opcoes = 0`: **bloqueia** até um filho terminar.
- Devolve o PID coletado ou `-1`.
- `EINTR`: interrompido por sinal, chame de novo.
- `ECHILD`: não há mais filhos.

Macro	Responde
<code>WIFEXITED(st)</code>	Terminou normalmente?
<code>WEXITSTATUS(st)</code>	Com que código?
<code>WIFSIGNALED(st)</code>	Morto por sinal?
<code>WTERMSIG(st)</code>	Qual sinal?

Zumbis e órfãos



Zumbi

Terminou, mas o pai ainda não coletou. Fica na tabela de processos (estado Z no ps) guardando o código de saída.

Órfão

O pai terminou antes. Outro processo adota e coleta: o PID 1 num servidor, ou o `systemd` `--user` numa sessão gráfica.

Por que coletar?

Um pai que nunca chama `waitpid` acumula zumbis.

E se o filho devolvesse o 30 no código de saída?

```
_exit(30);           /* no filho */  
WEXITSTATUS(status) == 30    /* no pai: funciona! */
```

E se o filho devolvesse o 30 no código de saída?

```
_exit(30);                /* no filho */  
WEXITSTATUS(status) == 30 /* no pai: funciona! */
```

Mas não serve como canal de dados:

- Só **8 bits**: valores de 0 a 255. `_exit(300)` chega como 44.
- Não cabe número negativo, `double`, texto ou vetor.
- Só chega quando o filho **termina**: um único valor, no fim.

Para que serve

Dizer *como* o processo terminou: 0 é sucesso, outro valor indica o tipo de falha.

1. Quantos processos são criados ao todo, incluindo o original? Desenhe a evolução por iteração.
2. Quais retornos de `fork()` identificam filho, pai e falha? De onde cada um continua?
3. O filho altera `v[i]`. Qual soma o pai imprime? Explique.
4. O pai espera os filhos. Isso faz o vetor dele receber as escritas dos filhos?

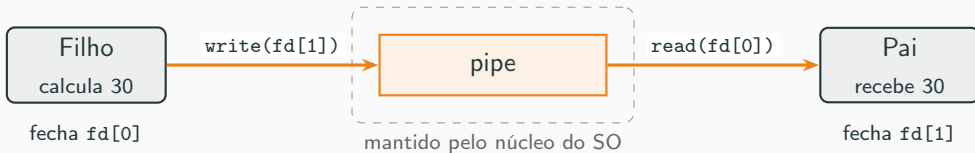
Vamos ouvir uma dupla sobre a árvore e outra sobre a espera.

Pipe: comunicação explícita

Se a memória não leva o resultado...

...o filho precisa **enviar** os dados explicitamente.

IPC (*Inter-Process Communication*): mecanismos do SO para processos trocarem dados.

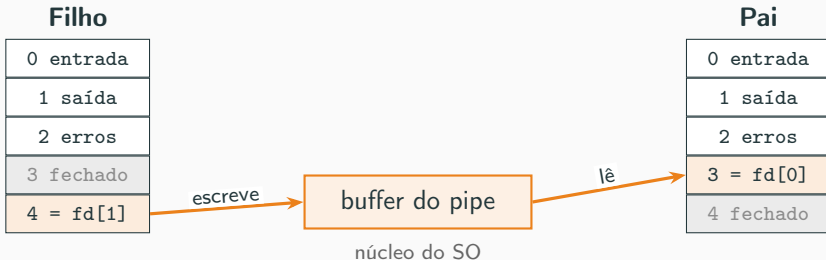


- `pipe(fd)` cria o canal: `fd[0]` é a ponta de **leitura**, `fd[1]` a de **escrita**.
- **Descritor**: número que identifica um recurso aberto pelo processo.
- O pipe é criado **antes** do `fork()`: pai e filho herdam descritores que referenciam o mesmo canal.
- Cada processo fecha a ponta que não usa.

Ordem que importa

`pipe()` → `fork()` → cada um fecha uma ponta → `write` / `read` → `waitpid`

A tabela de descritores depois do fork()



- 0, 1 e 2 já vêm abertos; por isso o pipe costuma receber 3 e 4.
- O `fork()` copia a tabela: as duas cópias apontam para o **mesmo** pipe do núcleo.
- Pipe criado **depois** do `fork()`: cada um teria o seu, sem canal em comum.

O exemplo pequeno: pipe_soma.c

```
if (pid == 0) {                                /* filho */
    close(fd[0]);
    valor = 10 + 20;
    int resultado = transferir(fd[1], &valor, sizeof valor, 1);
    close(fd[1]);
    _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
close(fd[1]);                                  /* pai */
int resultado = transferir(fd[0], &valor, sizeof valor, 0);
close(fd[0]);
```

transferir chama write (modo 1) ou read (modo 0) até mover todos os bytes.

Por dentro do helper transferir

```
while (tamanho > 0) {
    ssize_t n = modo ? write(fd, p, tamanho) : read(fd, p, tamanho);
    if (n < 0 && errno == EINTR) continue;      /* interrompido: tenta de novo */
    if (n < 0) return -1;                       /* erro de verdade */
    if (n == 0) return -1;                     /* canal fechado antes do fim */
    p += n;                                    /* avança no dado */
    tamanho -= (size_t)n;                      /* quanto ainda falta */
}
```

- read e write podem mover **menos bytes** do que o pedido; por isso o laço.
- Na prática vocês **usam** o helper; o foco é quem fecha qual ponta, quem envia e quem recebe.

O mesmo `&valor`, duas variáveis

- `&valor` no filho e `&valor` no pai apontam para variáveis **locais de cada processo**.
- O SO **copia os bytes** pelo canal.
- O ponteiro não torna as duas variáveis compartilhadas: o endereço é interpretado dentro do espaço de cada processo.

Pipe é fluxo de bytes

Nem toda chamada de `read` recebe tudo que pedimos. O *helper* transferir repete a operação e trata interrupções (`EINTR`).

E se o pai chegar ao read primeiro?

Situação do pipe	O que read faz
Vazio, com ponta de escrita aberta	Bloqueia e espera dados
Com dados disponíveis	Retorna os bytes (talvez menos que o pedido)
Vazio, todas as escritas fechadas	Retorna 0: EOF

Por isso fechamos pontas

Enquanto houver *qualquer* ponta de escrita aberta, inclusive uma cópia herdada esquecida, um pipe vazio não sinaliza EOF.

E do lado de quem escreve?

Situação	O que write faz
Há espaço no buffer do pipe	Copia os bytes e retorna.
Buffer cheio (64 KiB no Linux)	Bloqueia até o leitor consumir.
Nenhuma ponta de leitura aberta	Sinal SIGPIPE: encerra o processo. Se ignorado, -1 com EPIPE.

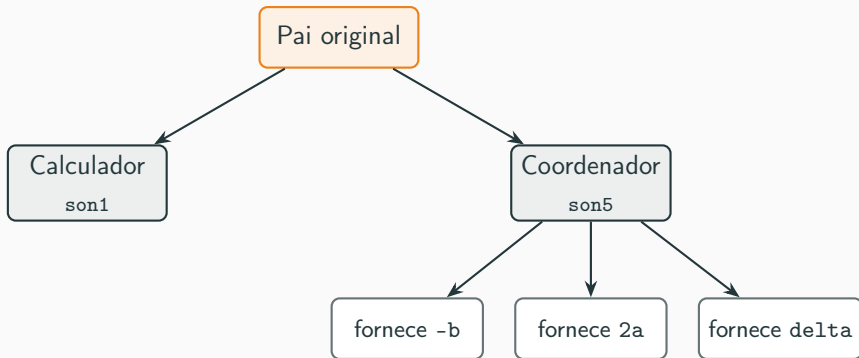
- Escritas de até PIPE_BUF bytes (4096 no Linux) são **atômicas**: não se misturam com as de outro processo.
- Um int cabe com folga.
- pipe2.c ignora SIGPIPE para poder tratar o erro em vez de morrer.

Executando o exemplo

```
$ make run-pipe-soma  
./build/pipe-soma  
Resultado recebido: 30
```

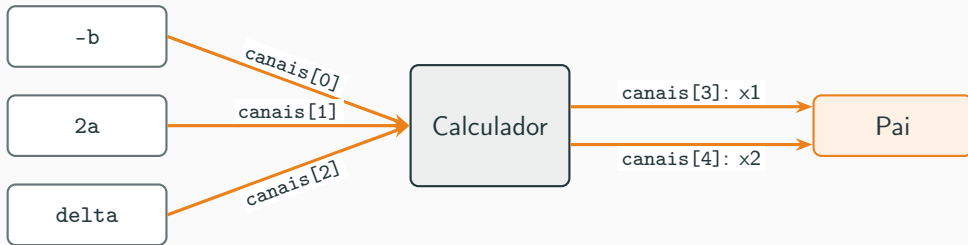
- O 30 foi calculado no filho.
- Chegou ao pai pelo canal, não por memória comum.

Vários processos: pipe2.c



Seis processos e cinco pipes. A entrada é lida no pai antes dos `fork()`.

pipe2.c: por onde os dados passam



Árvore de criação $\neq$ fluxo de dados

Os fornecedores são filhos do coordenador, mas enviam ao calculador. Quem escreve e quem lê cada canal?

pipe2.c: cada processo fica só com as pontas que usa

Processo	Lê de	Escreve em
Pai	canais[3], canais[4]	nenhum
Calculador	canais[0], [1], [2]	canais[3], [4]
Coordenador	nenhum	canais[0], [1], [2], só até criar os fornecedores
Fornecedor i	nenhum	canais[i]

fechar_exceto fecha todas as pontas, menos as indicadas. O coordenador guarda as de escrita só para os fornecedores herdarem, e logo depois fecha.

Por que tanto cuidado?

Uma ponta de escrita esquecida aberta impede o EOF: o calculador poderia esperar para sempre.

Executando pipe2.c

```
$ make run-pipe
./build/pipe2
>>> Digite tres numeros separados por espaco e tecle Enter: a b delta
>>> Exemplo: 1 -3 1    (delta ja calculado; o programa nao pede c)
a b delta: 1 -3 1
Raizes: x1 = 1; x2 = 2
```

Entrada: a, b e delta.

O programa **não** pede c.

$$-b = 3, \quad 2a = 2, \quad \sqrt{\Delta} = 1$$

$$x_1 = \frac{3-1}{2} = 1, \quad x_2 = \frac{3+1}{2} = 2$$

O exemplo ilustra IPC; não foi feito para acelerar um cálculo tão pequeno.

Prática

Seis lacunas: pipe_soma_atividade.c

```
if (pid == 0) {
    close(/* 1: ponta que o filho nao usa */);
    valor = /* 2: calculo */;
    int resultado = /* 3: enviar valor pelo helper transferir */;
    close(fd[1]);
    _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
close(/* 4: ponta que o pai nao usa */);
int resultado = /* 5: receber valor pelo helper transferir */;
close(/* 6: ponta usada pelo pai */);
```

transferir(descriptor, endereço, bytes, modo): modo 1 envia, modo 0 recebe.

Antes de preencher: perguntas que guiam cada lacuna

No filho

1. O filho só **escreve**. Qual ponta ele nunca usa?
2. Qual conta ele precisa fazer?
3. Envie valor: qual descritor, qual endereço, quantos bytes, qual modo?

No pai

4. O pai só **lê**. Qual ponta ele nunca usa?
5. Receba em valor: qual descritor, endereço, bytes e modo?
6. Terminou de ler: qual ponta falta fechar?

Confira

Endereço é `&valor` (não `valor`); tamanho é `sizeof valor`; `fd[0]` lê, `fd[1]` escreve.

Compilar e conferir

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    examples/dia05/pipe_soma_atividade.c -o /tmp/pipe-soma-aluno
$ /tmp/pipe-soma-aluno
Resultado recebido: 30
```

Em duplas, 7 minutos

- Antes de preencher, o esqueleto não compila.
- Cada pessoa explica uma ponta do canal.

Responda também

5. Por que criar o pipe antes do `fork()`?
6. `read` antes do `write`: erro, espera ou EOF?
7. Por que cada um fecha uma ponta diferente?

Fechamento

Se você pensou. . .	Reveja. . .
“O pai soma 18.”	Em qual processo cada atribuição acontece? Memórias separadas.
“São quatro processos.”	O filho não sai do laço: ele também chama <code>fork()</code> .
“ <code>waitpid</code> compartilha memória.”	Esperar o término não transporta bytes.
“ <code>read</code> sempre recebe tudo.”	O retorno diz quantos bytes chegaram.
“Pipe vazio sempre dá EOF.”	Ainda há alguma ponta de escrita aberta?
“Mesmo endereço, mesma memória.”	O endereço é virtual, interpretado em cada processo.
“O filho recomeça o <code>main</code> .”	Ele continua logo depois do <code>fork()</code> .

<code>fork()</code>	Cria um processo com memória independente .
<code>waitpid()</code>	Coordena o término e coleta o filho.
<code>pipe()</code>	Transporta dados entre processos.

Próxima aula: 7/10

Threads **compartilham** memória. O problema muda: como controlar o acesso a ela?

Individual, em duas ou três frases:

Por que `waitpid` não resolve a comunicação do resultado e `pipe` resolve?

Processo: programa em execução, com memória e recursos próprios.

PID / PPID: número do processo / do seu pai.

`fork()`: cria um filho, cópia do pai.

`waitpid()`: espera um filho terminar e o coleta.

Zumbi: terminou, mas ainda não foi coletado.

Órfão: o pai terminou antes; outro processo adota.

IPC: comunicação entre processos.

Pipe: canal de bytes de mão única, mantido pelo SO.

Descritor: número que o processo usa para um recurso aberto.

`fd[0]` / `fd[1]`: ponta de leitura / de escrita.

Bloquear: o processo dorme até a condição mudar.

EOF: `read` devolve 0; não há mais quem escreva.

Para estudar depois

Rode os exemplos (na raiz do repositório):

```
make run-fork-basico    make run-fork    make run-memoria  
make run-pipe-soma     make run-pipe    # digite 1 -3 1
```

Leia

- A **apostila** da aula: os mesmos temas, explicados com mais calma, e exercícios com respostas.
- `man 2 fork`, `man 2 waitpid`, `man 2 pipe`, `man 7 pipe`.
- Tanenbaum e Bos, *Sistemas Operacionais Modernos*, cap. 2.
- Silberschatz, Galvin e Gagne, *Fundamentos de Sistemas Operacionais*, cap. 3.

Dúvidas?

Weslei Ferreira Santos
MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Material-base: Prof. Maycon Leone M. Peixoto